

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



MIDDLEWARES E SANCTUM

Uso de Middlewares com Autenticação de SPAs e APIs via Cookies e Tokens com o Sanctum

Prof. Gill Velleda Gonzales

gillgonzales@ifsul.edu.br | [@gillgonzales](#) | [@g1ll](#)

CSTSI – IV Semestre – Desenvolvimento Back-End II

Novembro - 2025

API Token com Laravel Sanctum

Tópicos da Aula



- **Request Lifecycle e Middlewares**
 - Request Lifecycle
 - Middlewares
 - Sanctum: SPAs vs APIs
- **Modelo de Usuários**
 - Resource Controller de Usuários
 - Form Request para a criação de Usuários
- **Usando o Sanctum**
 - LoginController e RequestLogin
 - Criação do Token no Método Login
 - Configuração para Proteção das Rotas
- **Atividades**
- **Referências**

Middlewares e Request Lifecycle do Laravel

Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel



- Request Lifecycle

- Ponto de Entrada:

Todas as requisições são feitas para o ***public/index.php***, o servidor redireciona para este arquivo. Aqui são carregadas as classes do autoloader do composer e depois instanciada a aplicação a partir do ***bootstrap/app***.

- Kernel:

Logo as requisições são redirecionadas para o Console Kernel ou HTTP Kernel, dependendo do tipo de requisição; No HTTP Kernel são definidas as características de inicialização e os ***Middlewares*** que serão utilizados, como por exemplo o ***Middleware*** de **autenticação**.

Middlewares e Request Lifecycle do Laravel

Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel



- Request Lifecycle

- Service Providers:

Os *middlewares* geralmente são executados através de provedores de serviços (**Service Providers**); Mas outras funcionalidades do framework também são implementadas como serviços a serem utilizados durante a aplicação, como banco de dados, validação e rotas.

- Rotas:

O Serviço de Rotas é um dos Service Providers mais importantes do framework, **App\Providers\RouteServiceProvider**. Depois que todos os provedores de serviços forem configurados e registrados, o serviço de Rotas encaminha a Requisição para o método do controlador configurado no arquivo de rotas **routes/api.php**.

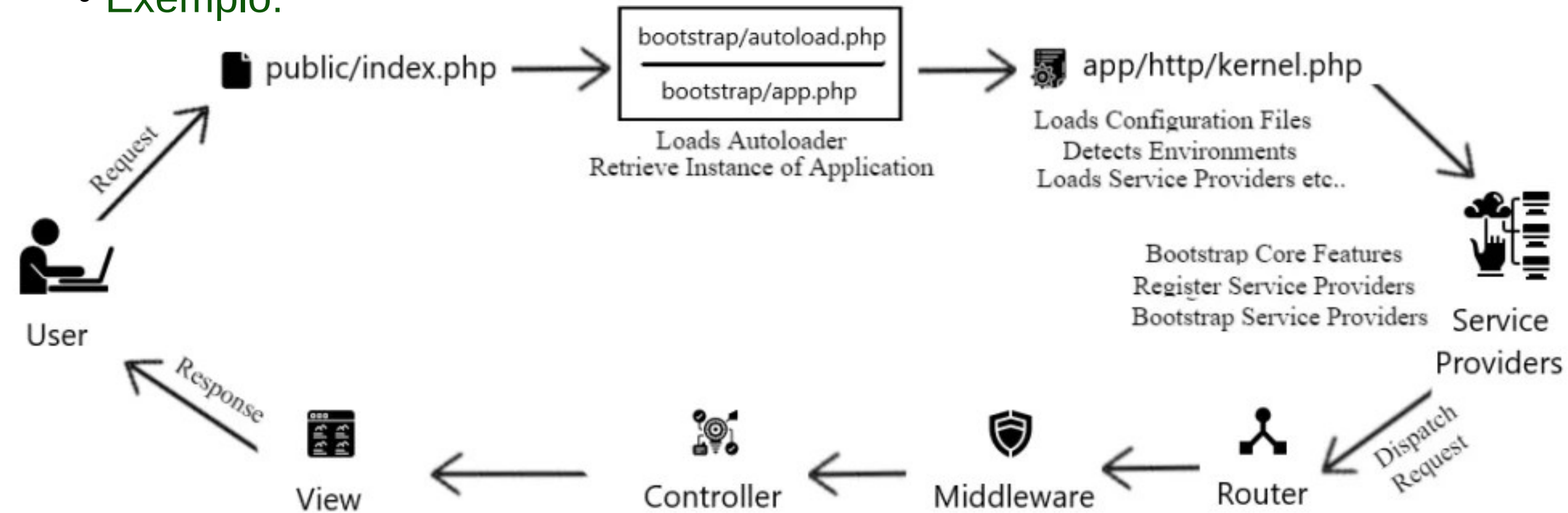
Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

- Request Lifecycle

- Exemplo:



FONTE: <https://dev.to/patelparixit07/laravel-request-lifecycle-195e>

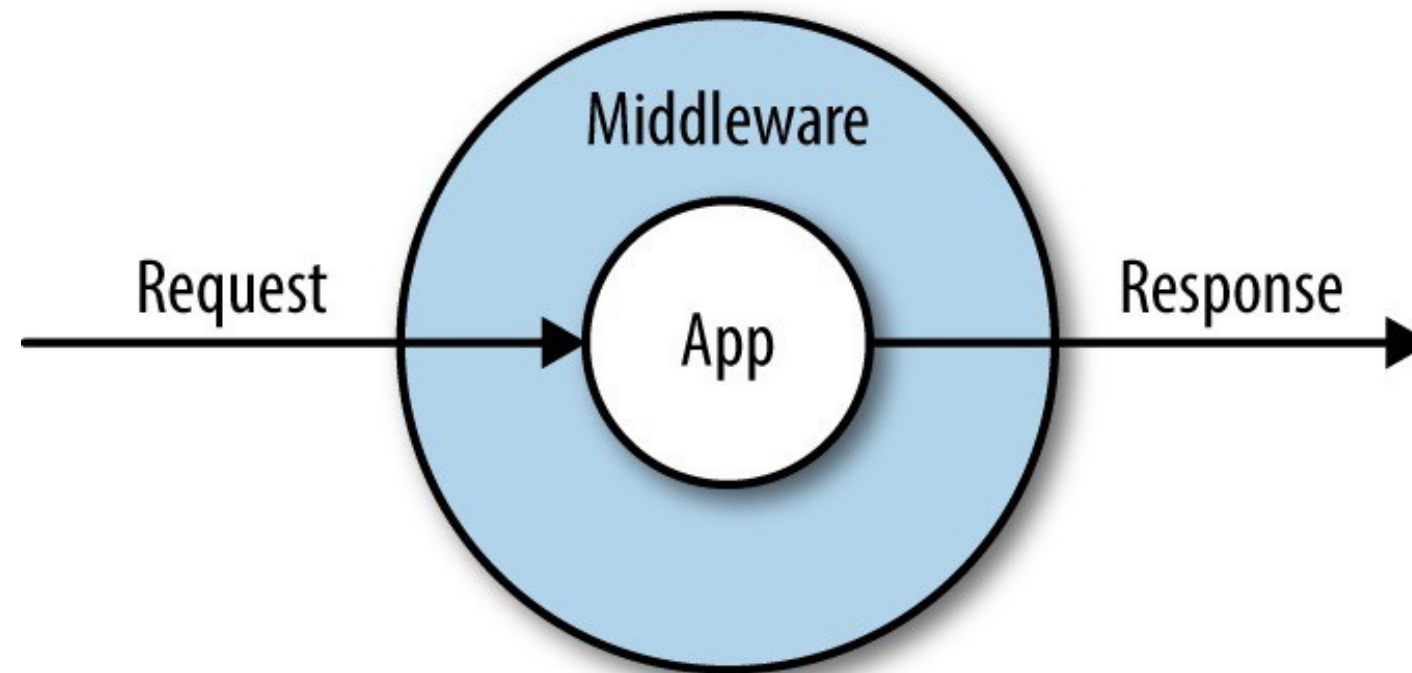
Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

- **Middleware**

- São os mecanismos de inspeção e filtro de requisições HTTP;
- A **autenticação** é feita por um middleware, responsável por verificar a requisição quanto a presença de credenciais (**cookies** ou **tokens**).
- Exemplo:



FONTE: <https://tony-stark.medium.com/laravel-5-8-middleware-tutorial-with-example-2019-9f5069b8c3cc>

Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

- **Middlewares:** podemos visualizar os middlewares do laravel usando a função ***getMiddlewareGroups()*** em ***withMiddleware*** nas configurações do ***app***.

```
bootstrap > app.php > Closure
```

```
3     ->withMiddleware(function (Middleware $middleware) {
4         dd($middleware->getMiddlewareGroups());
    }
```

```
array:2 [
  "web" => array:6 [
    0 => "Illuminate\Cookie\Middleware\EncryptCookies"
    1 => "Illuminate\Cookie\Middleware\AddQueuedCookiesToResponse"
    2 => "Illuminate\Session\Middleware\StartSession"
    3 => "Illuminate\View\Middleware\ShareErrorsFromSession"
    4 => "Illuminate\Foundation\Http\Middleware\ValidateCsrfToken"
    5 => "Illuminate\Routing\Middleware\SubstituteBindings"
  ]
  "api" => array:1 [
    0 => "Illuminate\Routing\Middleware\SubstituteBindings"
  ]
]
```


Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

• Middleware

- Para criar um **middleware** usamos o comando do artisan make:middleware:

```
php artisan make:middleware ForceJsonResponse
```

- Criamos uma nova classe de middleware na pasta **Http/Middleware** para forçar a resposta em JSON na nossa API.

```
PROTOTIPO-20242-LARAVEL-BACKEND-API
└─ app
   └─ Http
      └─ Controllers
      └─ Middleware
         └─ ForceJsonResponse.php
      └─ Requests
      └─ Resources
         └─ fornecedor
            ├── ProdutoCollection.php
            ├── ProdutoResource.php
            └── ProdutoStoredResource.php

app > Http > Middleware > ForceJsonResponse.php > PHP Intelephense > ForceJsonResponse > handle
8
9  class ForceJsonResponse
10 {
11     /**
12      * Handle an incoming request.
13      *
14      * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)
15      */
16     public function handle(Request $request, Closure $next): Response
17     { ...
20     }
21 }
22
```


Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

• Middleware

- No método ***handle*** de nosso ***middleware*** temos acesso a requisição pelo objeto **Request**.
- O nosso middleware, através do método **handle**, interceptará a requisição enviada a nossa rota e adicionará o **cabeçalho HTTP** (header) **Accept** com o valor **'application/json'**.
- Portanto, mesmo que o usuário esqueça de enviar este cabeçalho, nossa API sempre responderá em JSON em todas as rotas.

```
PROTOTIPO-20242-LARAVEL-B...  app > Http > Middleware > ForceJsonResponse.php > PHP Intelephense > ForceJsonResponse > handle
✓ app
  ✓ Http
    ✓ Controllers
    ✓ Middleware
      ForceJsonResponse.php
    > Requests
    ✓ Resources

9  class ForceJsonResponse
10
16  public function handle(Request $request, Closure $next): Response
17  {
18      $request->headers->set('Accept', 'application/json');
19      return $next($request);
20  }
21
22
```

Middlewares e Request Lifecycle do Laravel



Introdução aos Middlewares e Ciclo de Vida de Requisições no Laravel

- Mas para o nosso **middleware** funcionar em todas as rotas da API, precisamos registrá-lo na **pilha (stack) de middlewares** de apis.
- Observe que utilizamos o parâmetro **prepend**, para registrá-lo no início da pilha.
- O parâmetro **append** ou o método **appendToGroup('api')**, registram no final da pilha.
- Para visualizar a pilha de middlewares chamados nas rotas da api utilize o método **getMiddlewareGroups()**.

```
PROTOTIPO-20242-LARAVEL-B...  [Icons]
└─ app
└─ bootstrap
  > cache
  └─ app.php M
  └─ providers.php
└─ config
└─ database
└─ node_modules

bootstrap > app.php > Closure
8   return Application::configure(basePath: dirname(__DIR__))
9   >   ->withRouting( ...
14      )
15      ->withMiddleware(function (Middleware $middleware) {
16          $middleware->api(prepend: [
17              ForceJsonResponse::class,
18          ]);
19          // dd($middleware->getMiddlewareGroups());
20      })
```

Sobre o Sanctum

Introdução ao Sanctum, Middleware nativo para autenticação Token Simples



- O **sanctum** já está disponível quando instalamos o módulo de API do Laravel. (***php artisan install:api***)
- Provedor de Serviço de autenticação simplificada para SPAs (**S**ingle **P**age **A**pplication) e APIs.
- Autenticação para SPA (stateful): **NÃO USA TOKENS**, baseado em **COOKIES** e provem proteções adicionais como **CSRF**, **XSS**
- Autenticação para APIs: Utiliza **TOKENS** com **HABILIDADES** (ESCOPO);
 - Cada usuário da aplicação poderá gerar um ou vários API **Tokens** para suas contas.
 - Estes **Tokens** poderão possuir **habilidades específicas**, como **escopos** em sistemas de tokens mais complexos (**JWT**).
 - As **habilidades** definem quais **ações** dentro da aplicação um determinado token poderá dar **acesso** durante uma requisição.

Sobre o Sanctum

Introdução ao Sanctum, Middleware nativo para autenticação Token Simples



- Uso do **Sanctum** para **SPAs com Cookies HTTP Only**;
- Basicamente será a geração de um Cookie HTTP Only de autenticação mediante uma requisição à rota ***sanctum/csrf-cookie*** que retornará os cookies da sessão com proteção ***XSRF-TOKEN*** e ***laravel_session***.
- A partir deste momento, todas as requisições que incluam estes cookies, estarão protegidas e aplicação estará livre de ataques CSRF ou XSS.
- Exemplo dos ***cookies***:

Body Cookies (2) Headers (9) Test Results 204 No Content				
Name	Value	Domain	Path	Expires
XSRF-TOKEN	eyJpdil6lm5kL0Fsc1JOeGVVZn...	localhost	/	Tue, 20 May 20
laravel_session	eyJpdil6ljNtOG5FcVdDOHBJa1Z...	localhost	/	Tue, 20 May 20

- Várias configurações devem ser feitas para que estes cookies funcionem, como pode ser visto na **documentação**, além de **SPA** e **Stateful API** estarem no **MESMO DOMÍNIO TOP-LEVEL** (***spa.exemplo.dev***, ***api.exemplo.dev***);

Sobre o Sanctum

Introdução ao Sanctum, Middleware nativo para autenticação Token Simples



- Para o Uso do Sanctum como **StatefulAPI**, adicione o middleware **statefulApi** nas configurações de *middlewares* (*bootstrap/app.php*):

aula03 > bootstrap > app.php > Closure

```
15     ->withMiddleware(function (Middleware $middleware) {
20         // $middleware->api(prepend:ForceJsonResponse::class);
21         $middleware->api(prepend:ForceJsonResponse::class);
22         // dd($middleware->getMiddlewareGroups());
23         $middleware->statefulApi();
24     })
25     ->withExceptions(function (Exceptions $exceptions) {
26         //
27     })->create();
28
```

- Sempre revise a **documentação**, pode modificar dependendo da versão do *framework*;

Sobre o Sanctum

Introdução ao Sanctum, Middleware nativo para autenticação Token Simples



- Configurar no ENV as variáveis **SESSION_DOMAIN** e **SANCTUM_STATEFUL_DOMAIN** :

```
aula03 >  .env
```

```
34 SESSION_PATH=/
35 SESSION_DOMAIN=localhost
36 SANCTUM_STATEFUL_DOMAINS="localhost:8083"
37
```

- Além destas configurações é necessário sempre enviar o cabeçalho **Referer** e/ou **Origin** com a **URL** completa da aplicação.
- Sempre revise a **documentação**, pode modificar dependendo da versão do *framework*;

Sobre o Sanctum

Introdução ao Sanctum, Middleware nativo para autenticação Token Simples



- Uso do **Sanctum** para **APIs Token Authentication**;
- Basicamente será a geração de um Token de autenticação mediante uma requisição com **e-mail** e **senha** do usuário para uma **rota de login**.
- A aplicação então deverá responder com um **token** o qual deverá ser utilizado para acesso às rotas protegidas.
- Exemplo de **token**:

```
1 {  
2   "token": "15|QGdaOpSA8uPPuVpfJ0c4NdpCYwiwXVcRsJG6DEyC"  
3 }
```

- Este **token** deverá ser sempre enviado no cabeçalho de autenticação (**Authorization**) como um **bearer token** quando acessar uma rota protegida;

	KEY	VALUE	DESCRIPTION
☒	Authorization	Bearer 15 QGdaOpSA8uPPuVpfJ0c4NdpCYwi...	

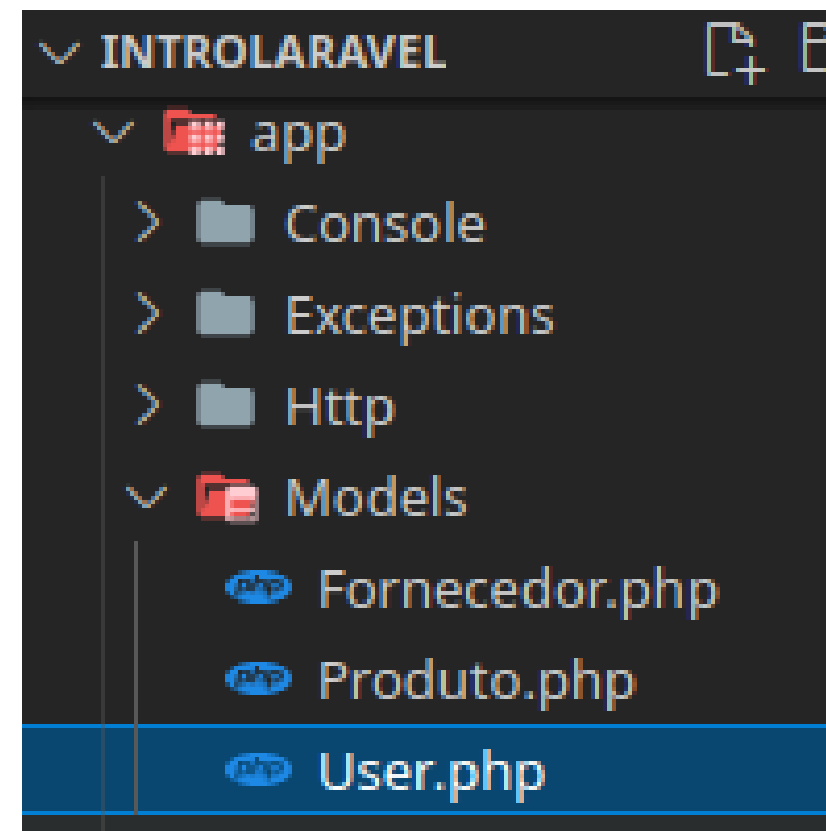
Usuários Autenticados

Criação do Modelo de Usuários (class User)



- **Model User**

- O Laravel já possui por padrão um modelo de usuário chamado **User**.
- Como podemos observar na pasta **app/Models**:
- Este modelo já implementa a autenticação.
- Por padrão o **User** usa **Sanctum** através do:
*use **HasApiTokens**.*
- Para trabalhar com este usuário vamos criar um **ResourceController** de **User**:
- Através deste controlador vamos implementar todos os métodos CRUD para o **User**.
- Principalmente para cadastro com o **store**.





- **Resource UserController**

- Para a criação do recurso do **Usuário**, vamos utilizar o artisan com os seguintes comandos:

```
php artisan make:controller -m User Api/UserController --resource --api
```

- O parâmetro **-m**, do comando acima, indica qual será o modelo utilizado pelo recurso, neste caso o **User**.
- Desta forma, o recurso já será criado utilizando o **Route Model Binding** nas requisições.
- Os parâmetros **--resource** e **--api** indicam a criação de um recurso de API.

```
echoes@echoes-Inspiron-15-3567 > ...Projetos/tsi_daoo/introLaravel > <?php
```

```
$ php artisan make:controller -m User Api/UserController --resource --api  
Controller created successfully.
```

Usuários Autenticados

Criação do do Controlador para Usuários



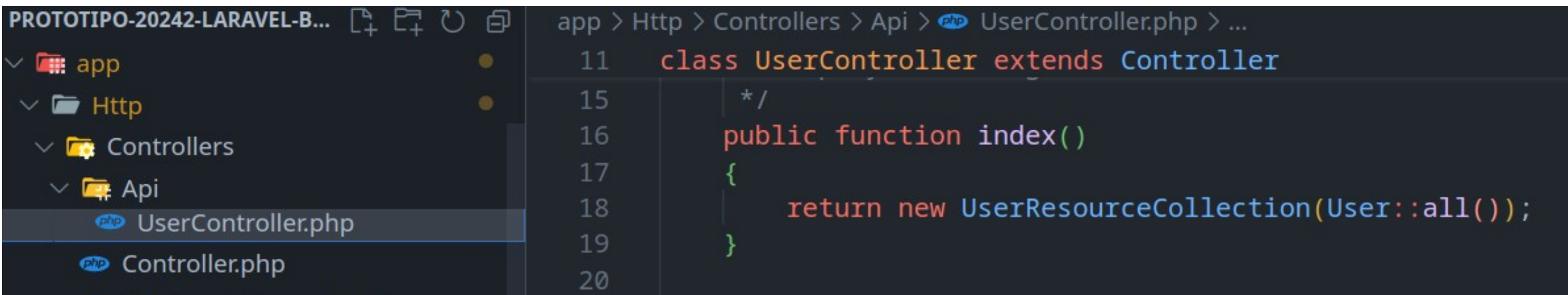
- **Resources JSON de Uses para o UserController**

- Criaremos os *ResourcesJson* de User para utilizar no *UserController*;

```
php artisan make:resource UserResource;
```

```
php artisan make:resource UserResourceCollection -c
```

- Criamos o método *index* retornando a coleção de usuários;
- No entanto, no momento não possuímos usuários na nossa base!
- Criaremos usuários aleatórios com os *Seeders* e *Factories*;



```
app > Http > Controllers > Api > UserController.php > ...
11  class UserController extends Controller
15      */
16      public function index()
17      {
18          return new UserResourceCollection(User::all());
19      }
20
```

Gerando Usuários com Factory e Seeder

Criação de Usuários fictícios no banco com o Factory



- **UserFactory**

- Para a criação dos primeiros Usuários no banco, vamos utilizar o **Factory** de Usuários;
- Já vem por padrão no Laravel o **UserFactory.php** na pasta **database/factories**;
- Inclusive os usuários a serem criados já possuem uma senha padrão;

database > factories > UserFactory.php > PHP Intelephense > UserFactory > definition

```
15 public function definition()
16 {
17     return [
18         'name' => $this->faker->name(),
19         'email' => $this->faker->unique()->safeEmail(),
20         'email_verified_at' => now(),
21         'password' => '$2y$10$92IXUNpkj00r0Q5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi', // password
22         'remember_token' => Str::random(10),
23     ];
24 }
25
```


Gerando Usuários com Factory e Seeder

Criação de Usuários fictícios no banco com o Factory



- **UserFactory**
- Inserimos os usuário com a classe **database/seeder/DatabaseSeeder**;
- Usaremos o factory de **User** para criar **3** usuários.

```
PROTOTIPO-20242-LARAVEL-B...  [+  +  ↺  ⌂]
└─ database
   └─ seeders
      DatabaseSeeder.php  M
      .gitignore
      database.sqlite
  > node_modules
  > public
  > resources
  > routes
     api.php  M

database > seeders > DatabaseSeeder.php > PHP Intelephense > DatabaseSeeder > run
9      class DatabaseSeeder extends Seeder
14         public function run(): void
15         {
16             User::factory(3)->create();
17
18             // User::factory()->create([
19             //     'name' => 'Test User',
20             //     'email' => 'test@example.com',
21             // ]);
22         }
23     }
```



- **Seed**

- Para efetivamente criar os usuários, deveremos executar a classe ***DatabaseSeeder.php***;
- Portanto rodaremos o comando do ***artisan*** para tal tarefa:
`php artisan db:seed`
- Com este comando criamos os ***Users*** no banco de dados.

```
echoes@echoes-Inspiron-15-3567 ...Projeto
ken x
$ php artisan db:seed
Database seeding completed successfully.

echoes@echoes-Inspiron-15-3567 ...Projeto
x
$
```



- **Rotas para UserController**

- Para testarmos o nosso controlador de Usuários, ***UserController***, devemos adicionar as ***Rotas*** para os métodos.
- Como estamos usando o conceito de ***Resource Controller*** do Laravel, poderemos chamar o método ***apiResource***;

```
11
12 Route::apiResource('produtos', ProdutoController::class);
13
14 Route::apiResource('users', UserController::class);
15
16
```



- Rotas para UserController

- Listando as Rotas de *users*:
- Para listar as rotas usamos o comando do artisan:

```
php artisan route:list --path=api/user
```

```
GET|HEAD api/user ..... users.index > Api\UserController@index
GET|HEAD api/users ..... users.store > Api\UserController@store
POST api/users ..... users.show > Api\UserController@show
GET|HEAD api/users/{user} ..... users.update > Api\UserController@update
PUT|PATCH api/users/{user} ..... users.destroy > Api\UserController@destroy
DELETE api/users/{user} .....
```

- A primeira **Rota** já está configurada por padrão no Laravel e serve para recuperar o usuário autenticado, verificando o **token**, veremos mais adiante:

```
8 Route::get('/user', function (Request $request) {
9     return $request->user();
10 }->middleware('auth:sanctum');
```

Criação de Usuários

Criação do UserRequest para validação de dados na criação de Usuários



- **UserRequest**

- Antes de implementarmos o método **store** em nosso controlador de Usuários, vamos criar a **FormRequest** de Usuário;
- Portanto rodaremos o seguinte comando no **artisan**:

```
php artisan make:request UserStoreRequest
```

- A saída do comando dever ser esta:

```
> php artisan make:request UserStoreRequest
```

```
INFO Request [app/Http/Requests/UserStoreRequest.php] created successfully.
```

- Agora precisamos criar no novo arquivo **UserRequest.php** as regras para a validação dos dados de **Usuário**.

Criação de Usuários

Criação do UserRequest para validação de dados na criação de Usuários



- **UserRequest**

- Criação das regras de validação dos dados do usuário:
 - **name** → Obrigatório e do tipo string até cinquenta caracteres;
 - **email** → Obrigatório do tipo e-mail e único, para cadastro no banco **users**
 - **password** → Obrigatório e no mínimo oito caracteres.

```
app > Http > Requests > UserRequest.php > PHP Intelephense > UserRequest > rules
24 public function rules()
25 {
26     return [
27         'name' => 'required | string | max:50',
28         'email' => 'required | email | unique:users',
29         'password' => 'required | min:8'
30     ];
31 }
32
```


Criação de Usuários

Criação do UserRequest para validação de dados na criação de Usuários



- **UserRequest**

- Personalização das mensagens de erro na validação do Usuário:

app > Http > Requests > UserRequest.php > PHP Intelephense > UserRequest > rules

```
33 public function messages()  
34 {  
35     return [  
36         'name.require'⇒'O nome é obrigatório!',  
37         'name.max'⇒'O nome deve ter no máximo 50 caracteres!',  
38         'email.required'⇒'O email é obrigatório!',  
39         'email.email'⇒'Informe um email válido!',  
40         'email.unique'⇒'Este email já está cadastrado!',  
41         'password.min'⇒'A senha deve ter no mínimo 8 caracteres!',  
42         'password.required'⇒'A senha é obrigatória !'  
43     ];  
44 }  
45  
46
```

Criação de Usuários

Criação do UserRequest para validação de dados na criação de Usuários



- **User**
 - A senha do usuário é automaticamente criptografada pela Model User no seu método **casts()** através do tipo hashed.
 - O **cast hashed** foi introduzido no Laravel 10.10 e não é mais necessário chamar as funções **bcrypt** ou **Hash::make()** para gerar a criptografia da senha antes de salvar no banco.
 - Agora este processo é automático.

```
11 class User extends Authenticatable
41     */
42     protected function casts(): array
43     {
44         return [
45             'email_verified_at' => 'datetime',
46             'password' => 'hashed',
47         ];
48     }
49 }
```

Registro de Usuários em UserController



Criação do método store para criar novo Usuário

- **Método `store()`** :criação de novos usuários pela rota **POST /api/users**;
- Usamos o **`UserRequest`** para **validar** os parâmetros de criação do **Usuário**.
- Acessamos os dados do usuário já validados com o método **`validated()`**.
- Repassamos os dados ao **`create`** da model de **User** para cadastro no banco.
- A senha será automaticamente criptografada pelo **cast `hashed`**.

aula03 > app > Http > Controllers > Api >  UserController.php > PHP Intelephense >  UserController >  store

```
14  class UserController extends Controller
27      public function store(UserStoreRequest $request)
28      {
29          try {
30              return (new UserResource(User::create($request->validated())))
31                  ->additional(['message' => 'Usuário criado com sucesso!!!']);
32          } catch (Exception $error) {
33              $this->errorHandler('Erro ao criar novo usuário!!!', $error);
34          }
35      }
36  }
```

Model de Usuários para Autenticação



Adaptação da model de usuário para o uso do sanctum

- **Propriedade password** : temporariamente, vamos alterar o model **User**, e trocar o campo **password** do array **\$hidden** para **\$fillable**;

```
app > Models > User.php > PHP Intelephense > User > $hidden
20 protected $fillable = [
21     'name',
22     'email',
23     'password'
24 ];
25
26 /**
27  * The attributes that should be hidden for serializati
28  *
29  * @var array<int, string>
30  */
31 protected $hidden = [
32     // 'password',
33     'remember_token',
34 ];
```

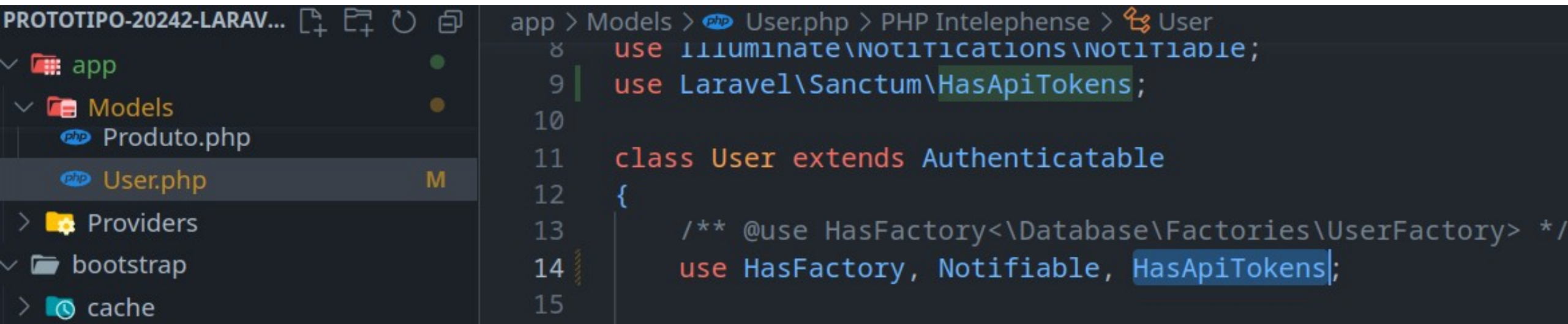

Model de Usuários para Autenticação

Adaptação da model de usuário para o uso do sanctum



- **Trait HasApiTokens:**

- Também devemos configurar na model de **User** a trait **HasApiTokens**;
- Através desta trait nossa model de usuário obterá acesso aos métodos de **criação de tokens** para o usuário autenticado.
 - **createToken()**
 - **plainTextToken**



```
app > Models > User.php > PHP Intelephense > User
8  use Illuminate\Notifications\Notifiable;
9  use Laravel\Sanctum\HasApiTokens;
10
11  class User extends Authenticatable
12  {
13      /** @use HasFactory<\Database\Factories\UserFactory> */
14      use HasFactory, Notifiable, HasApiTokens;
15  }
```

Criação de Usuários em UserController



Criação do método store para criar novo Usuário

- **Método *store()***

- Agora podemos criar um usuário acessando a **Rota */api/users*** com o verbo ***POST*** passando os parâmetros corretamente.

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/users/`. The 'Body' tab is selected, and the 'x-www-form-urlencoded' format is chosen. The request body contains the following data:

	KEY	VALUE	DESCRIPTION
☒	email	<code>gillgonzales@ifsul.edu.br</code>	
☒	password	<code>01234567</code>	
☒	name	<code>Gill Gonzales</code>	

Criação de Usuários em UserController

Criação do método store para criar novo Usuário



- Método *store()*

- Como resposta, se tudo ocorrer corretamente, teremos o novo Usuário.
- Notem que a senha foi criptografada, o algoritmo padrão do php ***bcrypt*** foi aplicado (***PASSWORD_DEFAULT***) através do facade ***Hash::make()***.
- Estamos vendo o campo ***password*** pois alteramos a model ***User***, removendo o campo ***password*** do atributo ***\$hidden***.

```
1  [
2      "mensagem": "Usuário cadastrado com sucesso!!",
3      "user": {
4          "email": "gillgonzales@ifsul.edu.br",
5          "password": "$2y$10$eWogvU32DNtbaSAB2RYXJ.GPuCopKvTYek0XqkMtavU7DE1ELu6Gy",
6          "name": "Gill Gonzales",
7          "updated_at": "2022-09-15T19:54:27.000000Z",
8          "created_at": "2022-09-15T19:54:27.000000Z",
9          "id": 4
10     }
11 ]
```

Classe LoginController



Responsável por verificar a autenticação

- **Camada Api/Auth:** Criaremos a classe **LoginController** e as demais classes filhas nas pastas **Api/Auth** e estenderemos a **Controller**.

```
sail artisan make:controller Api/Auth/LoginController
```

```
sail artisan make:controller Api/Auth/LoginController
```

INFO Controller [app/Http/Controllers/Api/Auth/LoginController.php] created successfully.

```
use App\Http\Controllers\Api\Controller;
use App\Models\User;
use Illuminate\Support\Facades\Auth;

class LoginController extends Controller
{
    public function authenticate(array $credentials): User | null
    { ...
    }
}
```

Método de Autenticação do LoginController



Validação dos parâmetros de Login com a classe Auth e Guard Web

- **Autenticação:** Implementamos a verificação do e-mail e senha através do método **attempt** da classe **Auth**. Se os dados estiverem corretos, retorna verdadeiro e a nossa função retorna a **model** do **usuário autenticado**.

```
class LoginController extends Controller
{
    public function authenticate(array $credentials): User | null
    {
        if (Auth::attempt($credentials))
            return User::where('email', $credentials['email'])->first();
        return null;
    }
}
```

- Mas no caso de erro, o **Auth** retorna **false** e retornamos **NULL** para quem chamou o **authenticate**.

Método de Autenticação do LoginController



Validação dos parâmetros de Login com a classe Auth e Guard Web

- **credentials:** para o método **attempt** funcionar, o parâmetro **\$credentials** deverá ser um **array** com os campos de e-mail e senha.

```
Illuminate\Support\Facades\Auth::attempt
<?php
public static function attempt($credentials) { }
[, bool $remember = false)
@param array $credentials
@return bool
(Auth::attempt($credentials))
```

- Nas classes de login filhas de **LoginController**, usaremos os dados validados de **LoginRequest** como **\$credentials**.

Form Request para Login

Validação dos parâmetros de Login com o LoginRequest



- **Criação do LoginRequest:**

- Para simplificar o método **login** da classe **LoginController**, podemos criar um **FormRequest** específico para o Login.
- Com o artisan, criaremos o **LoginRequest**.
- Desta forma, aparecerá na pasta **app/Http/Requests** o arquivo **LoginRequest.php**.
- Artisan:

```
php artisan make:request LoginRequest
```

```
> php artisan make:request LoginRequest
```

```
INFO Request [app/Http/Requests/LoginRequest.php] created successfully.
```

Form Request para Login

Validação dos parâmetros de Login com o LoginRequest



- **Criação do LoginRequest:**

- Adicionamos ao método **rules()** do **LoginRequest** as nossas regras:

app > Http > Requests > LoginRequest.php > PHP Intelephense > LoginRequest > rules

```
24 public function rules()
25 {
26     return [
27         'email'      => 'required | exists:users',
28         'password'   => 'required | string'
29     ];
30 }
31
```


Form Request para Login

Validação dos parâmetros de Login com o LoginRequest



- **Criação do LoginRequest**

- Adicionamos ao método **messages()** do **LoginRequest** as nossas mensagens personalizadas para os erros de validação:

app > Http > Requests > LoginRequest.php > PHP Intelephense > LoginRequest > messages

```
32     public function messages()
33     {
34         return [
35             'email.required' => 'O email é obrigatório !',
36             'email.exists'   => 'Email não cadastrado!',
37             'password.required' => 'A senha é obrigatória!'
38         ];
39     }
```

Login SPA (Cookies HTTP Only)

Criação do controlador de Login



- **A classe *LoginStatefullController*:** Estende a classe *LoginController*, utiliza a camada **Auth** e **LoginRequest** e retorna o user autenticado.

```
class LoginStatefullController extends LoginController
    public function login(LoginRequest $request): JsonResponse | JsonResponse
    {
        try {
            $credentials = $request->validated();
            $user = $this->authenticate($credentials);
            if (!$user)
                throw new Exception("Dados inválidos!");
            return new UserResource($user)
                ->additional(["message" => "Logado via sessão!!"]);
        } catch (Exception $error) {
            return $this->errorHandler(
```

Logout SPA (Cookies)

Criação do método logout utilizando o Guard Auth('web')



- Utilizamos a camada **Auth** para acessar a sessão aberta no **Guard Web** e realizar o logout, **invalidando** os **cookies HTTPOnly** de autenticação.

```
class LoginStatefullController extends LoginController

    public function logout(Request $request): JsonResponse
    {
        $user = Auth::guard('web')->user();
        if (!$user)
            return response()->json(["message" => "Não autenticado!"], 401);
        Auth::guard('web')->logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return response()->json(["message" => "Sessão encerrada!!!"]);
    }
}
```

- Os **guards** são configurados em **config/auth.php**.

Rotas de Login para SPA

Criação da Rota para efetuar o Login



- **Rota de Login direto na rota Web:**
 - Para a autenticação via Cookies **HTTPOnly**, mais recomendado para **SPAs**, registramos as rotas que usarão os métodos **login** e **logout** da classe **LoginStatefullController**, no arquivo **routes/web.php**:

aula03 > routes > web.php > Closure

```
1  <?php
2
3  use App\Http\Controllers\LoginWebController;
4  use Illuminate\Support\Facades\Route;
5
6  Route::controller(LoginWebController::class)->group(function(){
7      Route::post('/login','login');
8      Route::post('/logout','logout')->middleware("auth:sanctum");
9  });
```


Rotas de Login para SPA

Criação da Rota para efetuar o Login



- **Rota de Login na API para SPA:**

- No arquivo de rotas *routes/api.php*, criaremos a rota para o método *login* da classe *LoginStatefullController*:

outes >  api.php > ...

```
Route::prefix('v1')->group(function () {  
  
    Route::middleware('web')  
        ->prefix('spa')  
        ->controller(LoginStatefullController::class)  
        ->group(function () {  
            Route::post("/login", [LoginStatefullController::class, 'login']);  
            Route::post("/logout", [LoginStatefullController::class, 'logout']);  
        });  
});
```

- Como a rota foi configurada no arquivo *routes/api.php* precisamos adicionar o **middleware(web)**.

Criação do Login para a API com Tokens

Criação do controlador de Login



- **Login com Tokens**

- Iniciaremos criando o controlador com o artisan na camada **Api/Auth**:

```
sail artisan make:controller Api/Auth/LoginTokensController
```

- O comando criará o arquivo na pasta **app/Http/Controllers/Api/Auth**;

```
> sail artisan make:controller Api/Auth/LoginTokensController
```

```
INFO Controller [app/Http/Controllers/Api/Auth/LoginTokensController.php] created successfully.
```

- Estenderemos a classe **LoginController** e implementaremos o método **login**.
- O método Login será responsável por autenticar os dados do usuário e retornar o **Token** do **Usuário** autenticado;
- Usaremos o **Sanctum** para as funcionalidades de gerenciamento de **Tokens**.

Criação do Login da API com Tokens

Criação do controlador de Login



- **O método *login* em *LoginTokensController*:**

O método ***createToken*** é usado para criar o token e só está disponível no model ***User*** porque ele possui a Trait “***HasApiToken***” na sua classe. Para retornar o **token** na resposta acessamos o atributo ***plainTextToken***;

```
class LoginTokensController extends LoginController
{
    public function login(LoginRequest $request)
    {
        try {
            $user = $this->authenticate($request->validated());
            if (!$user) throw new Exception("Dados inválidos!");
            $token = $user->createToken($user->email)->plainTextToken;
            return compact('token', 'user');
        } catch (Exception $error) {
            return $this->errorHandler($error->getMessage(), $error, 401);
        }
    }
}
```

Revogação de Tokens - Logout

Implementação de logout com métodos de revogação



- Existem duas formas de se revogar os tokens gerados para um determinado usuário:

- ***tokens()*** → ***delete()***:

A função ***tokens()*** provém acesso a todos os tokens de um determinado usuário, combinado com o método ***delete()*** removemos todos os seus tokens.

- ***currentAccessToken()*** → ***delete()***:

Neste caso, com a função ***currentAccessToken*** teremos acesso apenas ao token utilizado durante a requisição, e poderemos revogar apenas este token com o método ***delete()***.

Rotas de Login para Tokens da API

Criação da Rota para efetuar o Login



- **Rota de Login na API para Tokens:**

- No arquivos de rotas, *routes/api.php*, criaremos a rota para o método *login* da classe *LoginTokensController*:

outes > api.php > Closure

```
Route::prefix('v1')->group(function () {  
  
    Route::prefix('tokens')  
        ->controller(LoginTokensController::class)  
        ->group(function () {  
            Route::post('logout', 'logout')->middleware("auth:sanctum");  
            Route::post('login', 'login');  
        });  
});
```

- Agora poderemos acessar esta **Rota** com os parâmetros de **e-mail** e **senha** e obter o token de autenticação (**Bearer**).

Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



- **Middleware em Rotas:**
- Para proteger uma rota específica, deveremos *atribuir* o *middleware* de autenticação.
- No caso deste exemplo, utilizamos o **Sanctum**, sistema nativo do **Laravel**.

aula03 > routes > api.php > ...

```
15 //todas as rotas protegidas (get, post, put, delete)
16 Route::apiResource('produtos', ProdutoController::class)
17     ->middleware('auth:sanctum');
18 Route::apiResource('users', UserController::class)
19     ->middleware('auth:sanctum');
20
```

- Desta forma, todas as **rotas** de **'produtos'** e **'users'** estarão protegidas e necessitarão de autenticação para serem acessadas.

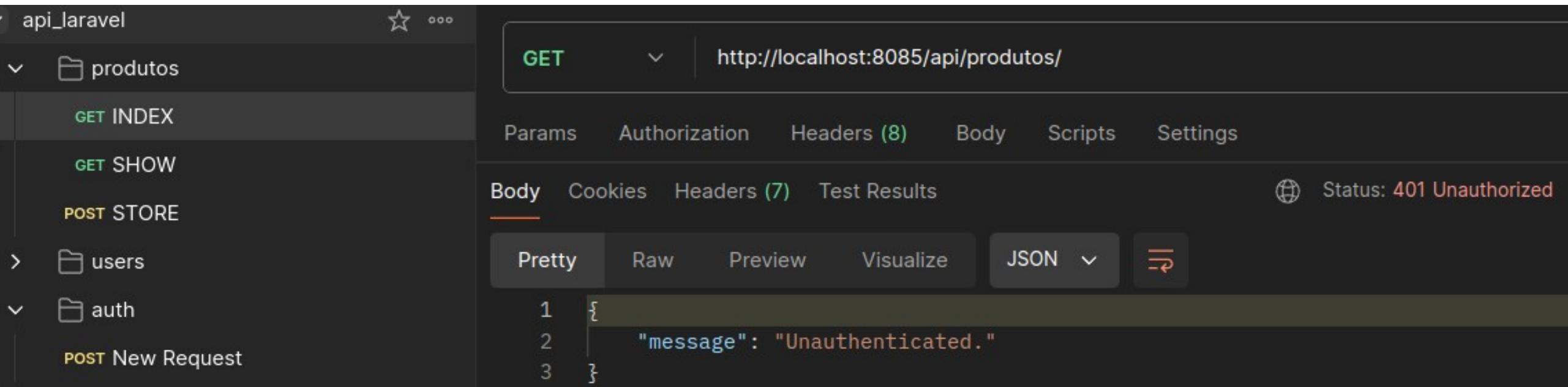
Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



Acessando rotas protegidas:

- Caso acessarmos a rota sem autenticação, receberemos um erro **401**.
- Lembrando que devemos passar o header ***Accept=application/json***;



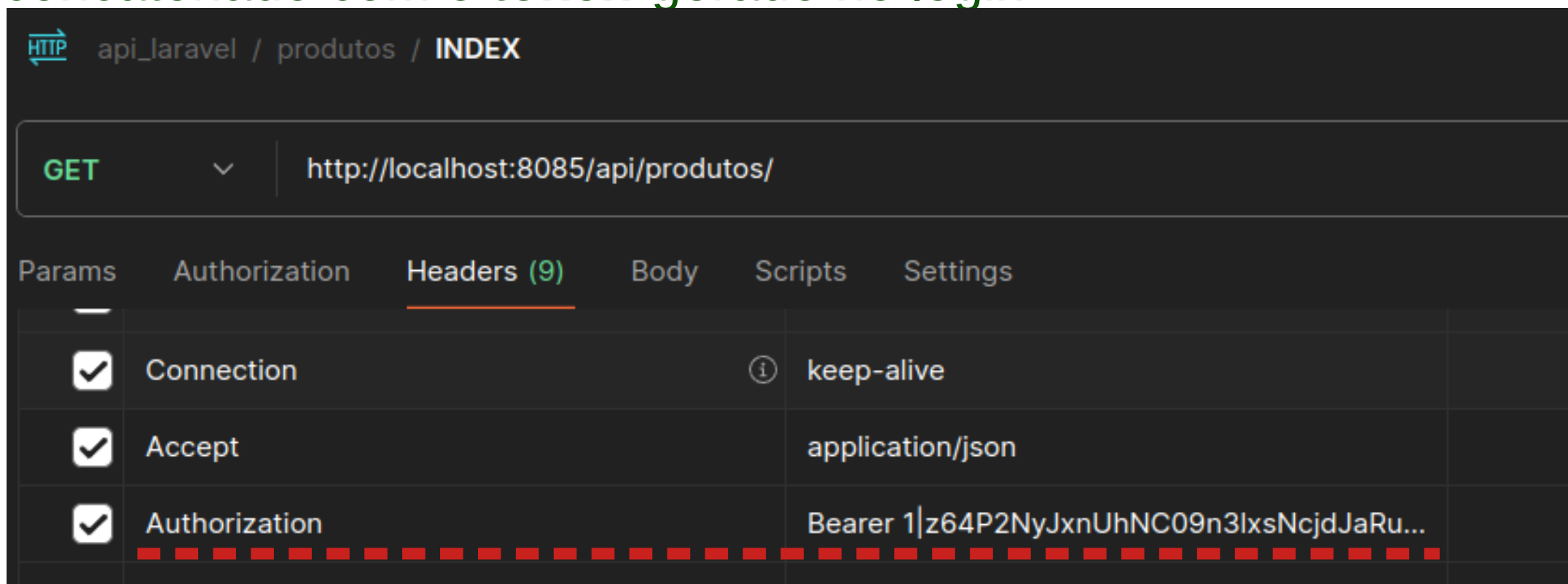
Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



Acessando rotas protegidas:

- Para ter acesso a mesma rota precisamos usar o **token**.
- No parâmetro HTTP **Authorization**, passamos o valor “***Bearer*** ” concatenado com o **token** gerado no login.



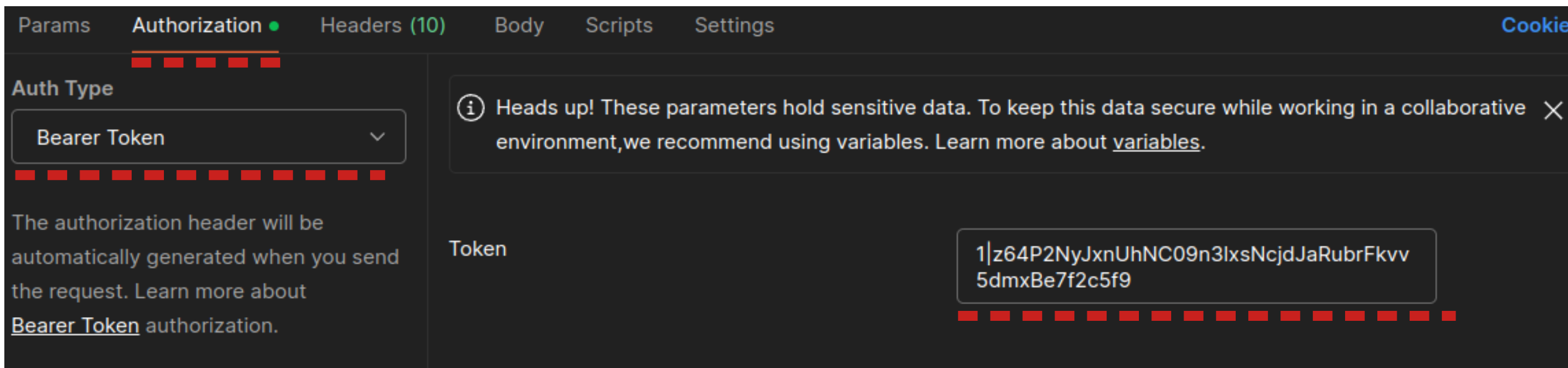
Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



Acessando rotas protegidas:

- No *Postman* é possível configurar este mesmo parâmetro na aba “**Authorization**”, escolher o método de autenticação (**Auth Type**) “**Bearer Token**” e salvar o valor do **token** no campo adequado.



Testando no Postman

Acessando a rota de Login no Postman para Tokens



DBE2-202501 - API Laravel V1 / auth / tokens / LOGIN

POST http://{{BASE_URL}}/api/v1/tokens/login

Params Authorization Headers (10) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☒ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

	Key	Value	Description	...	Bulk Edit
☒	email	admin@test.dev			
☒	password	adminadmin			

Body Cookies (2) Headers (7) Test Results 200 OK • 239 ms • 471 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "token": "5|8ATTbYlXGqV0eKQZ3PPjwTeEaI3WfXr0VI2MXl588caa106e",
3   "user": {
4     "id": 21,
5     "name": "Admin",
6     "email": "admin@test.dev",
7     "email_verified_at": "2025-11-01T20:03:55.000000Z",
8     "is_admin": 1,
9     "created_at": "2025-11-01T20:03:55.000000Z",
10    "updated_at": "2025-11-01T20:03:55.000000Z"
11  }
12 }
```

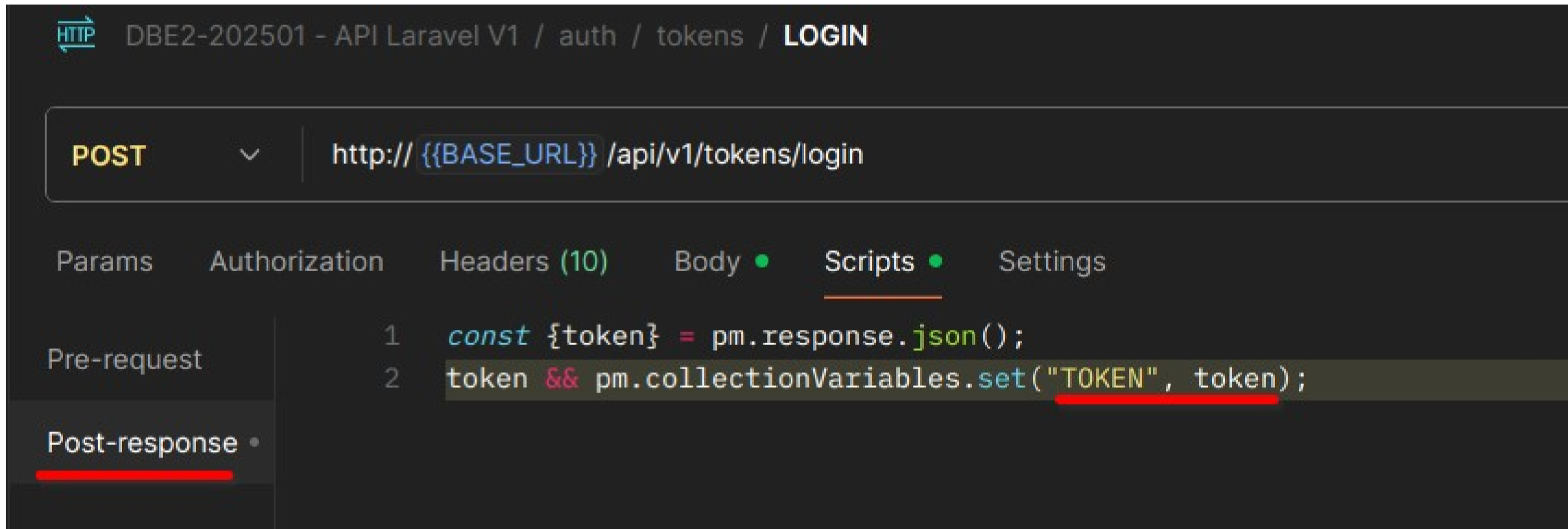
Testando no Postman

Acessando a rota de Login no Postman para Tokens



- **Scripts:**

- Neste exemplo usamos um script de pós-resposta para automatizar a configuração da variável de ambiente com o token **{{TOKEN}}**.

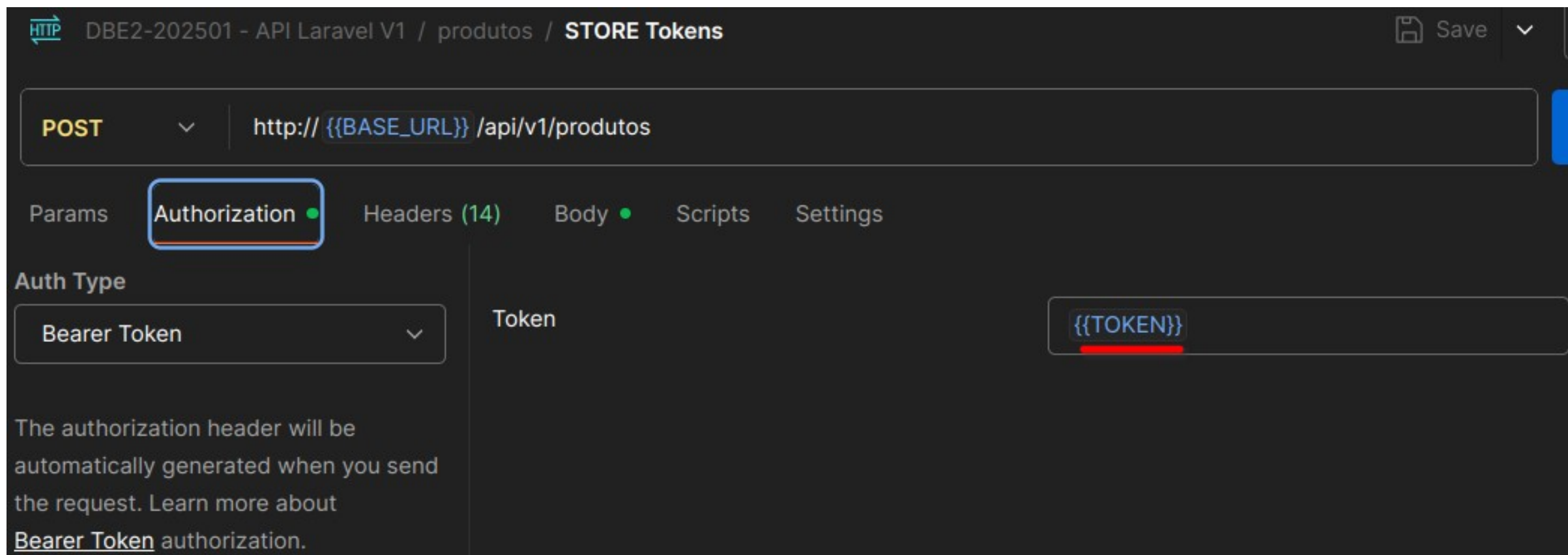


Testando no Postman

Acessando a rota de Login no Postman para Tokens



- ***Acessando uma rota protegida:***
 - Ao acessar a rota protegida POST de produtos, após o login, a variável **{{TOKENS}}** já estará preenchida com o novo token válido.



Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



- **Middleware em Rotas:**
- Para excluir as rotas **index** e **show** do recurso de produtos, vamos utilizar o método **only**, após o **apiResource**.

```
15 //rotas as rotas protegidas (get, post, put, delete)
16 Route::apiResource('produtos', ProdutoController::class)
17     ->middleware('auth:sanctum');
18
19 Route::apiResource('produtos', ProdutoController::class)
20     ->only(['index', 'show']); //sobrescreve a proteção de index e show
21
```

- Desta forma, serão recriadas apenas as rotas **index** e **show**, sem a proteção do **sanctum**, sobrescrevendo as rotas anteriores.
- Veja outros exemplos de configuração de rotas para o sanctum.

Proteção de Rotas

Protegendo rotas específicas com o Middleware Sanctum



- **Middleware em Rotas:**
- Outra forma de excluir as rotas desejadas do **middleware**, seria utilizar o método ***except*** para estas rotas.

```
22 Route::apiResource('users',UserController::class)->only(['store']);//D
23 Route::apiResource('users',UserController::class)
24     ->middleware('auth:sanctum')
25     ->except(['store']);//cadastro
26
```

- Desta forma, serão recriadas apenas as rotas ***show***, ***index***, ***update*** e ***delete***, com a proteção do ***sanctum***, sobrescrevendo as rotas desprotegidas definidas anteriormente.
- A rota post para ***store***, permanecerá **acessível** fora da proteção do sanctum.

Tempo de Expiração de um Tokens

Configuração do tempo de expiração de tokens no Sanctum



- **Expiração:**

- Por padrão os tokens do Sanctum nunca expiram e só podem ser destruídos pela revogação.
- No entanto, podemos configurar o Sanctum para expirar os tokens e determinar um tempo limite.
- No arquivo de configuração ***config/sanctum.php*** encontramos a variável ***expiration*** a qual podemos definir um valor em **MINUTOS**. O seu valor padrão é ***null***.

```
config > sanctum.php
45      */
46
47      'expiration' => 525600,
48
49      /*
```



1) Implemente a autenticação da sua API:

- Complete as rotas de autenticação com os métodos de Logout para tokens e crie uma opção para revogar todos os tokens ou apenas o enviado na requisição.
- Protejas as rotas de **POST**, **PUT** e **DELETE**. Apenas as rotas **GET** e de Login podem permanecer públicas (**EXCETO GET USERS**).
- Crie uma *branch* para a atividade de autenticação e envie o link do repositório remoto privado, com o professor como colaborador (**settings/colaborators**).
- Salve na pasta **database/dump** um dump do seu banco de dados com os dados. Mantenha a camada de modelo e **migrations** corretamente configuradas.
- Crie coleções de testes para as rotas de login criadas utilizando Cookies e Bearer Token com Variáveis de Ambiente.
- Exporte as coleções para a pasta **test/api** do seu projeto.



DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009

Fielding, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. Doctoral dissertation, University of California, Irvine, 2000.

Laravel Documentation – **Form Request Validation**, Disponível em: <https://laravel.com/docs/11.x/validation#form-request-validation>, Acesso em 20/10/2024.

Laravel Docs – **Middleware**, disponível em: <https://laravel.com/docs/11.x/middleware>, acesso em 16/11/2024.

Laravel Docs – **Request Lifecycle**, disponível em: <https://laravel.com/docs/11.x/lifecycle>, acesso em 16/11/2024.

Laravel Docs – **Laravel Sanctum**, disponível em: <https://laravel.com/docs/11.x/sanctum>, acesso em 15/11/2024.